SECURE CODING IN C/C++

Your Cheat Sheet

Be proactive

Waiting for something bad to happen before taking action is a costly and dangerous approach. Instead, aim to be proactive by understanding the most common vulnerabilities and their countermeasures. Taking this course is an excellent first step towards achieving this goal.

Review your code

Errors are inevitable, but they're also avoidable with a keen second pair of eyes. Frequent code reviews by team members well-versed in secure coding can be a strong line of defense against vulnerabilities.

Sharpen the saw

The world of security is ever-evolving, so your learning should be too. Stay in the loop with the latest security updates and trends to keep your coding skills relevant.



Greetings

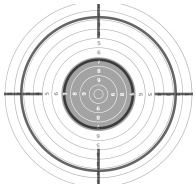
and welcome to your essential resource for secure coding in C and C++. The frightening possibility of a system crash at 30,000 feet serves as a wake-up call—when you're coding in powerful languages like C and C++, security should never be an afterthought; it's paramount.

With this guide, you'll have a concise yet comprehensive resource right at your fingertips.

Thank you

for investing your time to learn about secure coding in C/C++. I'm honored you chose my course to be part of your educational journey. Feel free to share this guide and the course link with others interested in upping their coding game.

Principles of Secure C and C++ Programming



Minimizing Attack Surface Area

Reducing the attack surface area is vital in enhancing the security of software. Attack surface area means the points in your code where an attacker may exploit vulnerabilities. By minimizing this area, we can lower the chances of successful attacks. In other words, the smaller the target, the harder it is to hit.

Input validation and sanitization, secure memory management, code reviews and testing and secure coding practices can help you reduce the attack surface area of your code.

Least Privilege



The Principle of Least Privilege is a key concept in secure programming. It suggests only granting the minimum necessary privileges required to do a specific task. By limiting access rights and privileges, the potential harm that can be caused by an attacker or compromised component is greatly reduced.

Examples include limiting code permissions, controlling system calls, and restricting access rights to the necessary minimum.



Fail-safe Defaults

The Principle of Fail-Safe Defaults suggests that access to resources and sensitive operations must be denied by default. Access should only be granted explicitly with permission.

To understand why this strategy is effective, consider the opposite scenario where access is granted by default but turned off if certain conditions are met. In such a case, there is a higher likelihood of overlooking critical conditions or failing to revoke access promptly, which can lead to potential security breaches.

Defense in Depth



To secure your C and C++ applications, relying on a single security mechanism isn't enough. A comprehensive approach is to use multiple layers of defense, known as "Defense in Depth." By designing your system this way, attackers will have to overcome several security measures, making it harder to compromise the system.

Common Security Pitfalls in C and C++ Programming

1 Buffer Overflows



One of the most common security flaws in C/C++ programs are buffer overflows. A buffer overflow occurs when a program writes more data to a buffer than it was designed to hold. Buffer overflows can happen on the stack, heap, or in a memory-mapped file.

🦄 Exploitability

Stack-based buffer overflows can be exploited to overwrite the return address on the stack, enabling an attacker to redirect the program flow to execute their own malicious code.

Heap overflows occur in dynamically allocated memory on the heap. Executing custom code is not usually possible with a heap overflow, but an attacker could still exploit this vulnerability by overwriting the data used by the application.

💥 Stack overflow example

```
// Triggering a stack-based buffer overflow
char buffer[5];
strcpy(buffer, "overflow");
```

✌️ Solution 1

Use the safer strncpy() function to avoid buffer overflows. Don't forget to add the null terminator because strncpy() won't add it in case of overflow.

```
strncpy(buffer, "overflow", sizeof(buffer) - 1);
// add terminating 0
buffer[sizeof(buffer) - 1] = 0;
```

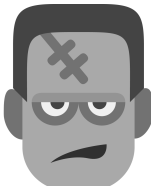
✌️ Solution 2

Alternatively, you can use snprintf() to avoid buffer overflows. This function will also add the null terminator automatically.

```
// adds the terminating zero
// even in case of overflows
snprintf(buffer, sizeof(buffer), "%s",
```

2 Risky Type Conversions

Despite using a static type system, C and C++ permit risky operations, such as implicit conversions and casting between incompatible types.



🦄 Exploitability

Misuse of these features can result in undefined behavior, potentially crashing the app or causing other unwanted side effects.

💥 Implicit type conversion example

In the following case, an implicit conversion happens from a double type to an integer type, leading to the loss of precision:

```
double x = 3.14;
int y = x;
```

✌️ Solution

Be explicit in your code and avoid implicit conversions to ensure type compatibility and prevent unintended consequences.

In this specific case, by using the static_cast operator, our code explicitly states that we perform a narrowing conversion on purpose.

```
// Explicit conversion from double to int
double a = 3.14;
int b = static_cast<int>(a);
```

Common Security Pitfalls in C and C++ Programming

3 Uncontrolled Format Strings



When a program uses an unchecked format string as an argument to the printf() command or similar functions, it can lead to the problem known as *uncontrolled format string,* a common issue that can compromise security.

Exploitability

An attacker could use format strings as arguments for printf() to reveal sensitive information stored in memory or execute arbitrary code.

Uncontrolled format string example

The user_input string may include format specifiers like %s or %d, printf() will try to use additional arguments that aren't there, resulting in undefined behavior.

And using the %x and %s format specifiers will cause printf() to print out values from the stack, revealing sensitive information.

The %n specifier commands printf() to write the number of characters successfully written so far to an address pointed to by the corresponding argument. As there's no corresponding argument, printf() will write to whatever location is at the top of the stack, leading to a potential security vulnerability.

```
// user_input may contain format specifiers
printf(user_input);
```

Solution

Using printf("%s", user_input) prevents any format specifiers in user_input from being interpreted by printf().

```
// format specifiers
printf("%s", user_input);
```

4 Improper Error Handling

Failing to handle errors properly can leave your code vulnerable to security risks and impact program reliability. Ignoring errors, failing to check the return values of functions that can fail, or exposing internal system details or valuable information in error messages are common mistakes in error management.



Exploitability

Subpar error handling might jeopardize the stability of your application, and revealing confidential data could offer hints to would-be attackers. Resource leaks could drain system capabilities and open the door to possible denial-of-service risks.

Ignored error example

In the following code, the return value of fopen() is not checked. If the file fails to open, the subsequent call to fscanf() will read data from an uninitialized or incorrect file stream, leading to undefined behavior:

```
char buffer[MAX_BUFFER_SIZE];
FILE *file = fopen("data.txt", "r");

fscanf(file, "%s", buffer);
cout << "First word: " << buffer << endl;

fclose(file);
```

Solution

The improved code handles the failed file opening scenario and logs the error:

```
char buffer[256];
FILE *file = fopen("data.txt", "r");

if (!file)
{
    printf("Failed to open the file.\n");
    return;
}

fscanf(file, "%s", buffer);
cout << "First word: " << buffer << endl;

fclose(file);
```

Common Security Pitfalls in C and C++ Programming

5 Race Conditions



A race condition occurs when two or more threads compete to access or modify shared data simultaneously. This can cause sporadic bugs and severe security vulnerabilities. Paying close attention to concurrency and handling it with care is crucial to avoid such risks.

Exploitability

Race conditions can cause data corruption, potentially leading to incorrect program behavior and security vulnerabilities such as unauthorized access to protected resources.

Race condition example

In this example, both threads access and modify the counter variable simultaneously without proper synchronization. As a result, the integrity of the counter variable is compromised, potentially leading to incorrect program behavior and other issues.

```
#include <iostream>
#include <thread>
using namespace std;

int counter = 0;

void increment_counter()
{
   ++counter;
}

int main()
{
    thread t1(increment_counter);
    thread t2(increment_counter);

    t1.join();
    t2.join();

    cout << "Counter value: " << counter << endl;

    return 0;
}
```

Solution

Introduce a mutex and use std::lock_guard to lock and unlock the mutex during the critical section. This way we ensure that only one thread can access and modify the counter variable at a time, preventing data corruption caused by a race condition.

```
#include <iostream>
#include <thread>
#include <mutex>
using namespace std;

int counter = 0;
mutex counterMutex;

void increment_counter()
{
    lock_guard<mutex> lock(counterMutex);
    ++counter;
}

int main()
{
    thread t1(increment_counter);
    thread t2(increment_counter);

    t1.join();
    t2.join();

    cout << "Counter value: " << counter << endl;

    return 0;
}
```

Common Security Pitfalls in C and C++ Programming

6 C/C++ Memory Management Mistakes



Unlike other languages, C and C++ do not have automatic garbage collection. Instead, developers must manage memory manually—a powerful but also risky feature. Thus, it's the responsibility of the programmer to. Failure to do so can result in vulnerabilities such as memory leaks, buffer overflows, and dangling pointers.

Exploitability

The side-effects of sloppy memory management can cause a wide range of problems with various degrees of severity, including:

- reduced system performance
- denial-of-service attacks due to resource exhaustion
- data corruption
- security vulnerabilities

Null pointer dereference

A null pointer is a pointer that does not point to any valid memory address. Dereferencing such a pointer can lead to undefined behavior and crashes.

```
int *ptr = NULL; // use nullptr in C++
// ... Perform some operations
*ptr = 10; // this will cause a runtime error
```

Solution

The simplest way to avoid dereferencing null pointers is to check whether the pointer is null before accessing its memory:

```
int *ptr = NULL; // use nullptr in C++
// ... Perform some operations
if(value != NULL) // use nullptr in C++
{
    *value = 5;
}
```

Referencing freed memory

In this code snippet, we first allocate memory to store an integer and save the address of this memory block in the variable value. Later, we free the allocated memory using the free() function.

Later, we attempt to write the value 5 to the same memory location that has already been freed. This can cause unpredictable behavior and data corruption, as the memory block may now be used for other purposes by the system.

```
int *value = malloc(sizeof(int)); // use new int()
in C++

free(value); // use operator delete in C++
// ... Perform some operations
// forget that value was freed and try to use it
*value = 5;
```

Solution

Implementing defensive coding practices, such as setting pointers to NULL can help reduce the risk of accessing freed memory.

Accessing a NULL pointer will instantly trigger a runtime error, which is far easier to debug and fix than unpredictable behavior resulting from dereferencing a dangling pointer. This approach turns a subtle bug into a blatant one, making it less likely to go unnoticed and thereby improving your application's overall security posture.

```
int *value = malloc(sizeof(int));
free(value); // use operator delete in C++
// Set value to NULL to make sure we don't use it
by mistake
value = NULL; // use nullptr in C++
// ... Perform some operations
*value = 5; // this will cause a runtime error
```

Accessing a NULL pointer will instantly trigger a runtime error, which is far easier to debug and fix than unpredictable behavior resulting from dereferencing a dangling pointer.

Wrapping Up

Consider these examples as just a snapshot of what you've already encountered in the course on Secure Coding in C/C++.

If you found these insights valuable and want to revisit any topics or perhaps share them with your network, the course is always there for you to go through again.

Feel free to share this guide and the course link with others interested in upping their coding game.

Thanks again for being a part of this learning journey with me. Take care and happy coding! 😊

Best regards,
Karoly Nyisztor

